

# République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



## Rapport AWS — Ubuntu EC2 et AWS CLI

Présenté par : **Ibrahima Khalilou Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**

Année académique 2025

# Sommaire

1. [Objectifs et contexte](#)
2. [Prérequis](#)
3. [Concepts et architecture](#)
4. [Installation et configuration](#)
5. [Réalisation pratique](#)
6. [Commandes et explications](#)
7. [Vérifications et tests](#)
8. [Dépannage](#)
9. [Sécurité et bonnes pratiques](#)
10. [Checklist](#)
11. [Conclusion](#)

# 1. Objectifs et contexte

Ce travail pratique présente la mise en place d'un serveur **Ubuntu Server** dans **Amazon EC2**, puis son administration avec l'**AWS Command Line Interface (AWS CLI)**. L'objectif est de comprendre le lien entre calcul, réseau, stockage, identité IAM et administration automatisable.

- Créer une instance EC2 à partir d'une AMI Ubuntu LTS.
- Configurer VPC, subnet, Security Group, stockage EBS et accès SSH.
- Installer AWS CLI v2 et vérifier son fonctionnement.
- Utiliser de préférence un rôle IAM attaché à l'instance.
- Valider la connectivité, l'identité AWS et les permissions.

**Principe :** EC2 fournit le calcul, Ubuntu fournit le système, VPC encadre le réseau, IAM contrôle les API AWS et AWS CLI fournit l'interface d'administration. Sur EC2, les identifiants temporaires d'un rôle d'instance sont préférables aux clés statiques.

## 2. Prérequis

### Ressources AWS

- Compte AWS actif ; MFA recommandée pour les identités humaines.
- Région choisie, par exemple `us-east-1`.
- Permissions EC2/VPC/IAM adaptées au TP.
- Paire de clés EC2 si SSH par clé est utilisé.

### Poste client

- Terminal Linux/macOS ou PowerShell avec OpenSSH.
- Accès Internet et clé privée `.pem` protégée.

| Élément      | Exemple                               |
|--------------|---------------------------------------|
| OS           | Ubuntu Server 24.04 LTS               |
| Architecture | x86_64 ou ARM64                       |
| SSH          | TCP/22 limité à l'IP d'administration |
| HTTP         | TCP/80 uniquement si nécessaire       |

### 3. Concepts et architecture

Poste d'administration | SSH TCP/22 ▼ AWS VPC → Subnet → EC2 Ubuntu → EBS | ↪  
Instance Profile / IAM Role → Services AWS

| Composant      | Fonction                               |
|----------------|----------------------------------------|
| AMI            | Modèle utilisé pour lancer le système. |
| EC2            | Instance de calcul exécutant Ubuntu.   |
| VPC/Subnet     | Isolation et adressage réseau.         |
| Security Group | Pare-feu stateful contrôlant les flux. |
| EBS            | Stockage bloc persistant.              |
| IAM Role       | Identité temporaire de l'instance.     |
| AWS CLI        | Client d'administration des API AWS.   |

**À retenir :** un Security Group ne remplace pas IAM : le premier contrôle le trafic réseau, le second les autorisations sur les API AWS.

## 4. Installation et configuration

### Lancement d'une instance Ubuntu EC2

**AMI**

Ubuntu Server LTS

**Stockage**

Volume EBS racine

**Calcul**

Type d'instance adapté au TP

**Identité**

IAM Role / Instance Profile

**Réseau**

VPC → Subnet → Security Group

**Validation**

Running + Status Checks

**Accès**

Key Pair + SSH TCP/22 limité

**Administration**

SSH puis AWS CLI

### 4.1 Création via la console EC2

1. Choisir **Ubuntu Server 24.04 LTS**.
2. Sélectionner une instance adaptée au TP.
3. Choisir/créer la Key Pair.
4. Choisir VPC et subnet.
5. Autoriser TCP/22 uniquement depuis l'IP d'administration.
6. Configurer le volume EBS et le tag **Name=Ubuntu-M2-UDB**.
7. Lancer l'instance et relever son IP publique si SSH direct est utilisé.

### 4.2 Connexion SSH

```
chmod 400 mykey.pem ssh -i mykey.pem ubuntu@PUBLIC_IP
```

Le compte initial d'une image Ubuntu EC2 est généralement **ubuntu**. La clé privée ne doit jamais être publiée.

### 4.3 Préparation du système

```
sudo apt update sudo apt upgrade -y sudo reboot
```

## 5. Réalisation pratique

### Connexion SSH – Ubuntu EC2

```
$ chmod 400 mykey.pem  
$ ssh -i mykey.pem ubuntu@PUBLIC_IP  
The authenticity of host ... can't be established.  
Are you sure you want to continue connecting? yes  
  
Welcome to Ubuntu Server LTS  
ubuntu@ip-10-0-1-25:~$  
  
SSH opérationnel – serveur prêt à administrer
```

### 5.1 Diagnostic initial

```
hostnamectl cat /etc/os-release uname -m ip addr ip route df -h free -h
```

### 5.2 Installation d’AWS CLI v2

AWS fournit un installateur Linux officiel. Pour une installation système :

```
sudo apt update sudo apt install -y curl curl -fsSL
```

```
https://awscli.amazonaws.com/v2/install.sh | sudo bash -s -- --system aws --version
```

Pour une installation reproductible avec contrôle de version, l’archive officielle [awscliv2.zip](#) peut être utilisée. Adapter l’archive à `x86_64` ou `aarch64` selon `uname -m`.

### 5.3 Identité IAM de l’instance

Attacher un IAM Role au profil d’instance permet à AWS CLI de récupérer des identifiants temporaires sans stocker de clés permanentes.

```
aws sts get-caller-identity aws configure set region us-east-1 aws ec2 describe-  
instances --max-results 5
```

**À éviter :** mettre une Access Key dans Git, un script ou une image EC2. Utiliser un rôle d’instance ou, lorsque nécessaire, un profil IAM géré selon le moindre privilège.

### 5.4 Création EC2 par CLI — exemple

```
aws ec2 run-instances \ --image-id ami-XXXXXXX \ --instance-type t3.micro \ --key-  
name ma-cle-ec2 \ --security-group-ids sg-XXXXXXX \ --subnet-id subnet-XXXXXXX \ --  
tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Ubuntu-M2-UDB}]'
```

`--image-id` identifie l'AMI, `--instance-type` définit les ressources, tandis que subnet et Security Group déterminent le rattachement réseau.

## 6. Commandes et explications

| Commande                                        | Utilité            | Point important                                  |
|-------------------------------------------------|--------------------|--------------------------------------------------|
| <code>ssh -i mykey.pem ubuntu@IP</code>         | Connexion distante | <code>-i</code> sélectionne la clé privée.       |
| <code>aws --version</code>                      | Version CLI        | Valide l'installation.                           |
| <code>aws sts get-caller-identity</code>        | Identité effective | Diagnostic l'authentification.                   |
| <code>aws configure set region us-east-1</code> | Région par défaut  | Évite de répéter <code>--region</code> .         |
| <code>aws ec2 describe-instances</code>         | Interroger EC2     | <code>--max-results</code> limite les résultats. |
| <code>systemctl status ssh</code>               | État SSH           | Vérifie le service systemd.                      |
| <code>journalctl -u ssh</code>                  | Logs SSH           | <code>-u</code> sélectionne l'unité.             |

### Commandes de diagnostic AWS

```
aws configure list aws sts get-caller-identity aws ec2 describe-regions --query 'Regions[0:3].[RegionName,OptInStatus]' --output table
```

## 7. Vérifications et tests

### Validation AWS CLI et identité IAM

```
$ aws --version
aws-cli/2.x Python/3.x Linux/x86_64

$ aws sts get-caller-identity
{
  "UserId": "...",
  "Account": "...",
  "Arn": "arn:aws:iam::...:role/EC2Role"
}
```

Identité temporaire du rôle EC2 validée

### 7.1 Système

```
systemctl is-system-running systemctl is-active ssh ss -lntp | grep ':22' df -h /
free -h
```

### 7.2 Réseau

```
ip route getent hosts aws.amazon.com curl -I https://aws.amazon.com
```

### 7.3 AWS CLI et IAM

```
aws --version aws sts get-caller-identity aws ec2 describe-instances --max-results 5
```

**Validation attendue :** SSH fonctionne, le service est actif, DNS/HTTPS répondent lorsque le réseau le permet, AWS CLI s'exécute et les appels AWS autorisés retournent des données.

## 8. Dépannage

| Problème                            | Diagnostic                               | Correction                                                              |
|-------------------------------------|------------------------------------------|-------------------------------------------------------------------------|
| SSH timeout                         | IP, route, subnet, SG                    | Vérifier TCP/22, IP cible et routage.                                   |
| Permission denied (publickey)       | Utilisateur et clé                       | Utiliser <code>ubuntu</code> , la bonne clé et <code>chmod 400</code> . |
| <code>aws: command not found</code> | <code>command -v aws</code>              | Vérifier installation et PATH.                                          |
| AccessDeniedException               | <code>aws sts get-caller-identity</code> | Corriger uniquement la policy IAM nécessaire.                           |
| Could not resolve host              | <code>getent hosts aws.amazon.com</code> | Vérifier DNS, route et configuration VPC.                               |
| Pas d'Internet sortant              | <code>ip route</code>                    | Vérifier route table et Internet Gateway/NAT.                           |
| Disque plein                        | <code>df -h</code>                       | Nettoyer ou augmenter EBS.                                              |

```
sudo journalctl -u ssh --no-pager -n 100 sudo systemctl status ssh sudo dmesg | tail -n 50
```

## 9. Sécurité et bonnes pratiques

- Activer MFA sur les identités humaines.
- Appliquer le moindre privilège IAM.
- Préférer les rôles IAM et identifiants temporaires sur EC2.
- Ne jamais publier clés AWS ou clés SSH privées dans Git.
- Limiter TCP/22 à un réseau d'administration connu.
- Privilégier Systems Manager Session Manager lorsque possible afin d'éviter une exposition SSH directe.
- Maintenir Ubuntu et les composants à jour.
- Chiffrer les volumes EBS et maîtriser leur cycle de vie.
- Superviser l'instance et conserver les journaux utiles.
- Arrêter/supprimer les ressources de TP inutilisées pour éviter les coûts.

**Production :** une instance privée administrée par Systems Manager et protégée par une architecture réseau contrôlée est généralement préférable à une instance exposant SSH directement sur Internet.

# 10. Checklist

## Validation du TP

- ☐ Région AWS choisie
- ☐ AMI Ubuntu LTS sélectionnée
- ☐ Instance EC2 créée et démarrée
- ☐ Key Pair protégée
- ☐ Security Group configuré au moindre privilège
- ☐ Connexion SSH fonctionnelle
- ☐ Ubuntu mis à jour
- ☐ Architecture CPU vérifiée
- ☐ AWS CLI v2 installée
- ☐ Région CLI configurée
- ☐ IAM Role d'instance configuré si nécessaire
- ☐ `aws sts get-caller-identity` validé
- ☐ Appel EC2 testé
- ☐ Tests DNS/HTTPS réalisés
- ☐ Procédure de dépannage comprise
- ☐ Ressources inutilisées arrêtées ou supprimées

# 11. Conclusion

Ce travail pratique a permis de mettre en œuvre une chaîne complète allant du lancement d'un serveur Ubuntu sur Amazon EC2 jusqu'à son administration avec AWS CLI. La démarche couvre les dimensions système, réseau, stockage, identité et automatisation nécessaires à l'exploitation d'une instance cloud.

La séparation des responsabilités est essentielle : **EC2** fournit le calcul, **VPC/Security Groups** encadrent le réseau, **IAM** contrôle les autorisations et **AWS CLI** fournit l'interface d'administration.

